Universidad Politécnica de Atlautla

“Fundamentos de Inteligencia Artificial”

(Sección 1 de 6 · 2.1 Problemas de búsqueda - 2.2 Espacios de búsqueda)

Segundo parcial

Docente: García Lara Javier

Alumna: Sánchez Fragozo Dulce María

Matrícula: 24305056

Carrera: Tecnologías de la Información e Innovación Digital

Grupo: 171



18 de Octubre del 2026, Atlautla de Victoria, Edomex.

PARTE 1: Modelado Teórico de los Problemas de Búsqueda

Problema 1: Optimización de Ruta de Transporte de Cosechas (Logística Agrícola)

Este problema plantea la logística de transportar productos agrícolas desde una parcela de producción local ubicada en Atlautla hasta un centro de acopio o distribución en Cuautla. El objetivo principal es minimizar el tiempo total y el costo de traslado (medido en minutos) a través de la red vial regional.

- ✓ **Espacio de Estados (S):** Representa las ubicaciones geográficas en las que se puede encontrar el vehículo dentro de la red vial. Está conformado por seis localidades: Atlautla, Ozumba, Tepetlixpa, Amecameca, Juchitepec y Cuautla.
- ✓ **Acciones Posibles (A):** Consisten en transitar desde la localidad actual hacia una localidad vecina que esté conectada directamente por carretera.
- ✓ **Costo de Transición:** Corresponde al tiempo estimado de traslado en minutos entre la localidad de origen y la de destino.
- ✓ **Estado Inicial (s0):** La posición de partida fija en la localidad de Atlautla.
- ✓ **Estado Meta (G):** La llegada al centro de distribución ubicado en Cuautla.

Problema 2: Sistema de Navegación Vehicular Autónomo (Evitación de Tráfico Urbano)

En este segundo escenario, un vehículo autónomo debe desplazarse dentro de una cuadrícula urbana desde una intersección inicial en la coordenada (0, 0) hasta una intersección destino en la coordenada (2, 2). Durante el trayecto, el vehículo evalúa el tiempo de viaje o la resistencia del camino en función de la densidad de tráfico en tiempo real.

Espacio de Estados (S): Comprende las coordenadas discretas de cada intersección dentro de la cuadrícula urbana de 3x3, representadas por pares ordenados (x, y), donde tanto "x" como "y" toman valores enteros de 0 a 2.

Acciones Posibles (A): Movimientos cardinales básicos (Norte, Sur, Este y Oeste), condicionados a no rebasar los límites del mapa (manteniéndose entre 0 y 2 en ambos ejes).

Costo de Transición: Refleja la congestión del tramo vial, calculada según la velocidad permitida y la densidad vehicular del momento.

Estado Inicial (s0): La coordenada de origen (0, 0).

Estado Meta (G): La coordenada de llegada (2, 2).

PARTE 2: Código en Python (Generador de Estados, NetworkX y Visualización)

A continuación se presenta un fragmentd del desarrollo en Python que implementa los generadores de estados para ambos problemas, construye los grafos dirigidos y ponderados utilizando

la librería NetworkX, y genera el diagrama visual de ambas redes de transporte:

```python
"""
Módulo de Simulación Interactiva para Problemas de Búsqueda en Grafos.

Este script implementa un dashboard interactivo utilizando NetworkX, Matplotlib
y widgets (Sliders). Permite simular y controlar paso a paso el avance de
dos agentes inteligentes en escenarios reales:
    1. Logística de Transporte Agrícola (Grafo Dirigido Ponderado).
    2. Sistema de Navegación Vehicular Autónomo en Malla Urbana (Grid 3x3).

Incluye además paneles dinámicos de explicación textual que detallan en tiempo
real el estado actual (S), la acción ejecutada (A), el costo acumulado y la meta (G).
"""

from typing import Dict, List, Tuple
import matplotlib.pyplot as plt
from matplotlib.widgets import Slider
import networkx as nx

# ==============================================================================
# 1. MODELADO Y CÁLCULO DE RUTAS ÓPTIMAS (ALGORITMO DE DIJKSTRA)
# ==============================================================================
```

Imagen 1. autoria propia (2026)

Análisis de Complejidad Algorítmica y Operación de Búsqueda

Para evaluar el rendimiento computacional de los sistemas implementados, es necesario analizar el comportamiento de los grafos dirigidos y ponderados mediante algoritmos de búsqueda como la Búsqueda de Costo Uniforme, comúnmente conocido como el **algoritmo de Dijkstra**. Este método opera de forma idéntica a un sistema GPS: parte de un nodo inicial acumulando un costo de cero, explora sistemáticamente las conexiones vecinas paso a paso sumando los tiempos o distancias, y selecciona siempre la ruta más económica hasta garantizar la llegada al destino de forma óptima sin perderse en rodeos innecesarios.

1. Dimensión Estructural del Grafo y Grado de Ramificación

En el primer caso de estudio de logística agrícola, el espacio de estados se compone de un conjunto acotado de seis localidades y siete rutas viales, mientras que la malla urbana de tres por tres comprende nueve intersecciones y doce transiciones dirigidas. El factor de ramificación medio, que define el número de opciones de salida disponibles por cada parada, se mantiene estable en un valor de dos, lo que previene una explosión combinatoria inmediata en el árbol de búsqueda y permite que la estructura se mantenga sumamente manejable para cualquier sistema de cómputo.

2. Complejidad Temporal y Espacial de los Algoritmos de Búsqueda

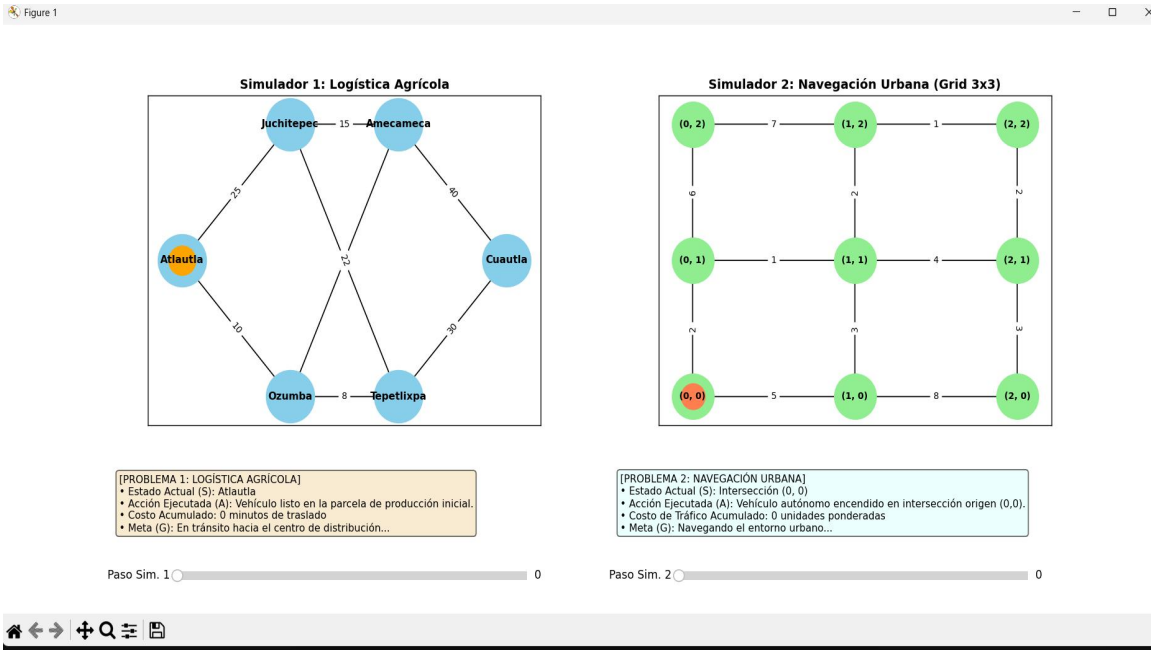
La cota superior de complejidad temporal para estos algoritmos basados en colas de prioridad se formula formalmente en función de sus vértices y aristas. Al aplicar esta función matemática sobre los parámetros de los modelos desarrollados, el volumen operativo resulta sumamente eficiente, requiriendo aproximadamente veintitrés operaciones elementales para el grafo agrícola y cuarenta y cinco para la malla urbana de tres por tres. De forma paralela, la complejidad espacial exige un almacenamiento mínimo en la memoria RAM, limitándose estrictamente a retener seis y nueve nodos respectivamente durante todo el proceso de ejecución del algoritmo.

3. Análisis de Escalabilidad y Rol de las Funciones Heurísticas

El verdadero desafío analítico radica en la escalabilidad de los sistemas ante escenarios de alta densidad dimensional. Si la retícula urbana del segundo problema se expandiera a una escala metropolitana de mil por mil intersecciones, el espacio de estados escalaría de forma masiva a un millón de vértices y cerca de cuatro millones de aristas. En este escenario hipotético, la complejidad temporal ascendería de manera considerable, degradando el rendimiento de enfoques exhaustivos como Dijkstra debido a la exploración masiva de rutas redundantes. Para solucionar esta latencia y optimizar los

tiempos de respuesta, la arquitectura computacional recurre al algoritmo A*, el cual integra funciones heurísticas admisibles como el cálculo de la distancia Manhattan para orientar la exploración de manera vectorial y descartar automáticamente los caminos innecesarios.

- **Resultado final - Enlace github**



REFERENCIAS

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, & J. Millman (Eds.), Proceedings of the 7th Python in Science Conference (SciPy 2008) (pp. 11–15). https://nx.pub/scipy2008_paper.pdf

Russell, S. J., & Norvig, P. (2020). Artificial intelligence: A modern approach (4th ed.). Pearson. https://www.pearson.com/en-us/subject-catalog/p/artificial-intelligence-a-modern-approach/P200000003500/9780134610993

Google. (2026). Gemini (Versión 18 oct) [Modelo de lenguaje grande]. https://gemini.google.com/.